

Documentation Technique

Application de gestion de collections personnelles

1. Architecture générale

L'application repose sur une architecture web classique en trois couches :

- une base de données relationnelle MySQL (collection_perso) qui stocke les types de collections, les collections, les éléments et les emprunts ;
- une couche d'accès aux données en PHP, utilisant l'extension PDO (PHP Data Objects) pour communiquer avec la base de manière sécurisée ;
- une couche de présentation, constituée de pages PHP mêlant traitement serveur et affichage HTML.

Chaque fonctionnalité de l'application correspond à un script PHP autonome (collections.php, ajout_collection.php, elements.php, ajout_element.php, emprunts.php), qui inclut systématiquement le fichier de connexion connexion.php.

2. Base de données

2.1 Schéma relationnel

La base de données collection_perso est composée de quatre tables liées entre elles par des clés étrangères :

Table	Clé primaire	Clé étrangère	Référence
types_collection	id_type	—	—
collections	id_collection	id_type	types_collection
elements_collection	id_element	id_collection	collections
emprunts	id_emprunt	id_element	elements_collection

Remarque : la relation est en cascade : un type possède plusieurs collections, une collection possède plusieurs éléments, et un élément peut faire l'objet de plusieurs emprunts successifs.

2.2 Script de création des tables

```
CREATE DATABASE collection_perso;
USE collection_perso;

CREATE TABLE types_collection (
  id_type INT AUTO_INCREMENT PRIMARY KEY,
  nom_type VARCHAR(100) NOT NULL
);

CREATE TABLE collections (
  id_collection INT AUTO_INCREMENT PRIMARY KEY,
  nom_collection VARCHAR(150) NOT NULL,
  id_type INT NOT NULL,
  FOREIGN KEY (id_type) REFERENCES types_collection(id_type)
);
```

```
CREATE TABLE elements_collection (
    id_element INT AUTO_INCREMENT PRIMARY KEY,
    titre_element VARCHAR(150) NOT NULL,
    numero INT,
    possede BOOLEAN DEFAULT 1,
    id_collection INT NOT NULL,
    FOREIGN KEY (id_collection) REFERENCES collections(id_collection)
);

CREATE TABLE emprunts (
    id_emprunt INT AUTO_INCREMENT PRIMARY KEY,
    nom_emprunteur VARCHAR(100),
    date_emprunt DATE,
    date_retour DATE,
    id_element INT NOT NULL,
    FOREIGN KEY (id_element) REFERENCES elements_collection(id_element)
);
```

3. Connexion à la base de données (PDO)

La connexion à la base est centralisée dans le fichier connexion.php, inclus au début de chaque script ayant besoin d'accéder aux données. Elle utilise l'extension PDO, qui offre une couche d'abstraction orientée objet et un support natif des requêtes préparées.

```
<?php
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=collection_perso;charset=utf8",
        "root",
        ""
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(Exception $e) {
    die("Erreur : " . $e->getMessage());
}
?>
```

Le DSN (Data Source Name) `mysql:host=localhost;dbname=collection_perso;charset=utf8` indique au driver PDO le type de base (MySQL), le serveur (localhost), la base à utiliser (collection_perso) et l'encodage des caractères (utf8). Les deux paramètres suivants correspondent aux identifiants de connexion (utilisateur root, mot de passe vide en environnement de développement). Le mode d'erreur `PDO::ERRMODE_EXCEPTION` permet de lever une exception en cas d'échec d'une requête plutôt que de poursuivre silencieusement l'exécution.

4. Scripts et traitements

4.1 Affichage des collections — collections.php

Ce script affiche la liste des collections existantes, en associant chaque collection à son type grâce à une jointure SQL :

```
SELECT c.nom_collection, t.nom_type
FROM collections c
JOIN types_collection t ON c.id_type = t.id_type
```

Le résultat est parcouru avec une boucle foreach et affiché dans un tableau HTML. La fonction htmlspecialchars() est appliquée à chaque donnée affichée afin de neutraliser les caractères spéciaux et de prévenir les failles XSS.

4.2 Ajout d'une collection — ajout_collection.php

Ce script affiche un formulaire (nom de la collection + liste déroulante des types) et traite sa soumission :

```
if($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = $_POST['nom'];
    $type = $_POST['type'];
    $sql = "INSERT INTO collections (nom_collection, id_type) VALUES (?, ?)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$nom, $type]);
    echo "Collection ajoutée";
}
```

La liste déroulante des types est alimentée dynamiquement à partir de la table types_collection (SELECT * FROM types_collection), afin de rester synchronisée avec les données réellement présentes en base.

Remarque : la requête d'insertion utilise une requête préparée avec des marqueurs positionnels (?), ce qui protège le script contre les injections SQL en séparant la structure de la requête des données fournies par l'utilisateur.

4.3 Affichage des éléments — elements.php

Ce script liste l'ensemble des éléments enregistrés, avec leur titre, leur numéro, le nom de leur collection et leur statut de possession. Il nécessite une jointure entre elements_collection et collections :

```
SELECT e.titre_element, e.numero, c.nom_collection, e.possede
FROM elements_collection e
JOIN collections c ON e.id_collection = c.id_collection
```

Le champ possede, de type BOOLEAN, est affiché sous forme de texte lisible («Oui» / «Non») grâce à un opérateur ternaire côté PHP : \$row['possede'] ? "Oui" : "Non".

4.4 Ajout d'un élément — ajout_element.php

Ce script affiche un formulaire permettant de saisir le titre et le numéro d'un nouvel élément, ainsi qu'une liste déroulante pour choisir la collection de rattachement :

```
if($_SERVER["REQUEST_METHOD"] === "POST") {
    $titre = $_POST['titre'];
    $numero = $_POST['numero'];
    $collection = $_POST['collection'];
    $sql = "INSERT INTO elements_collection
        (titre_element, numero, id_collection)
        VALUES (?, ?, ?)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$titre, $numero, $collection]);
    echo "Élément ajouté";
}
```

```
<form method="post">
```

```
Titre : <input type="text" name="titre"><br>
Numéro : <input type="number" name="numero"><br>
Collection :
<select name="collection">
  <?php foreach($collections as $c): ?>
    <option value="<?= $c['id_collection'] ?>">
      <?= htmlspecialchars($c['nom_collection']) ?>
    </option>
  <?php endforeach; ?>
</select>
<button type="submit">Ajouter</button>
</form>
```

Comme pour l'ajout de collection, la liste déroulante est générée dynamiquement à partir de la table collections, afin d'éviter toute saisie libre incohérente avec les données existantes.

4.5 Gestion des emprunts — emprunts.php

Ce script doit permettre d'enregistrer et de consulter les prêts effectués. La table emprunts associe un élément (id_element) à un emprunteur (nom_emprunteur), une date de prêt (date_emprunt) et une date de retour (date_retour). Un emprunt est considéré comme en cours lorsque date_retour est NULL :

```
SELECT e.nom_emprunteur, el.titre_element, e.date_emprunt
FROM emprunts e
JOIN elements_collection el ON e.id_element = el.id_element
WHERE e.date_retour IS NULL
```

5. Sécurité mise en œuvre

- Requêtes préparées (PDO::prepare / execute) sur toutes les requêtes d'insertion, afin de prévenir les injections SQL.
- Échappement systématique des données affichées avec htmlspecialchars(), afin de prévenir les failles XSS.
- Gestion centralisée des erreurs de connexion via PDO::ERRMODE_EXCEPTION et un bloc try/catch.

Remarque : en environnement de production, les identifiants de connexion (utilisateur, mot de passe) ne doivent pas rester en dur dans connexion.php mais être externalisés dans un fichier de configuration non versionné.

6. Limites et pistes d'évolution

- Absence de gestion des utilisateurs : l'application ne distingue pas plusieurs comptes, toutes les collections sont partagées.
- Absence de contrôle de saisie côté serveur (vérification des champs obligatoires, format des dates pour les emprunts).
- Mot de passe de connexion à la base vide, à corriger avant toute mise en production.
- Le script emprunts.php reste à finaliser : formulaire d'enregistrement d'un prêt et mise à jour de la date de retour.